



Workpath SDK DeviceInfo Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	DeviceInfo Sample on a device	4
3	DeviceInfo Sample Flow	5
4	Initializing SDK.....	6
5	Checking DeviceService/DeviceSettingsService Capability	8
6	Getting device attributes	9
7	Getting list of attributes	11
8	Enabling/Disabling ports for print	12
8.1	Disable Port Permission	13
8.2	Deprecated enableExternalPrinting() API	13
9	Legal notice	14
10	Support.....	15

1 Introduction

DeviceInfo Sample provides use case scenario how to read various attributes from a device.

1.1 Document conventions

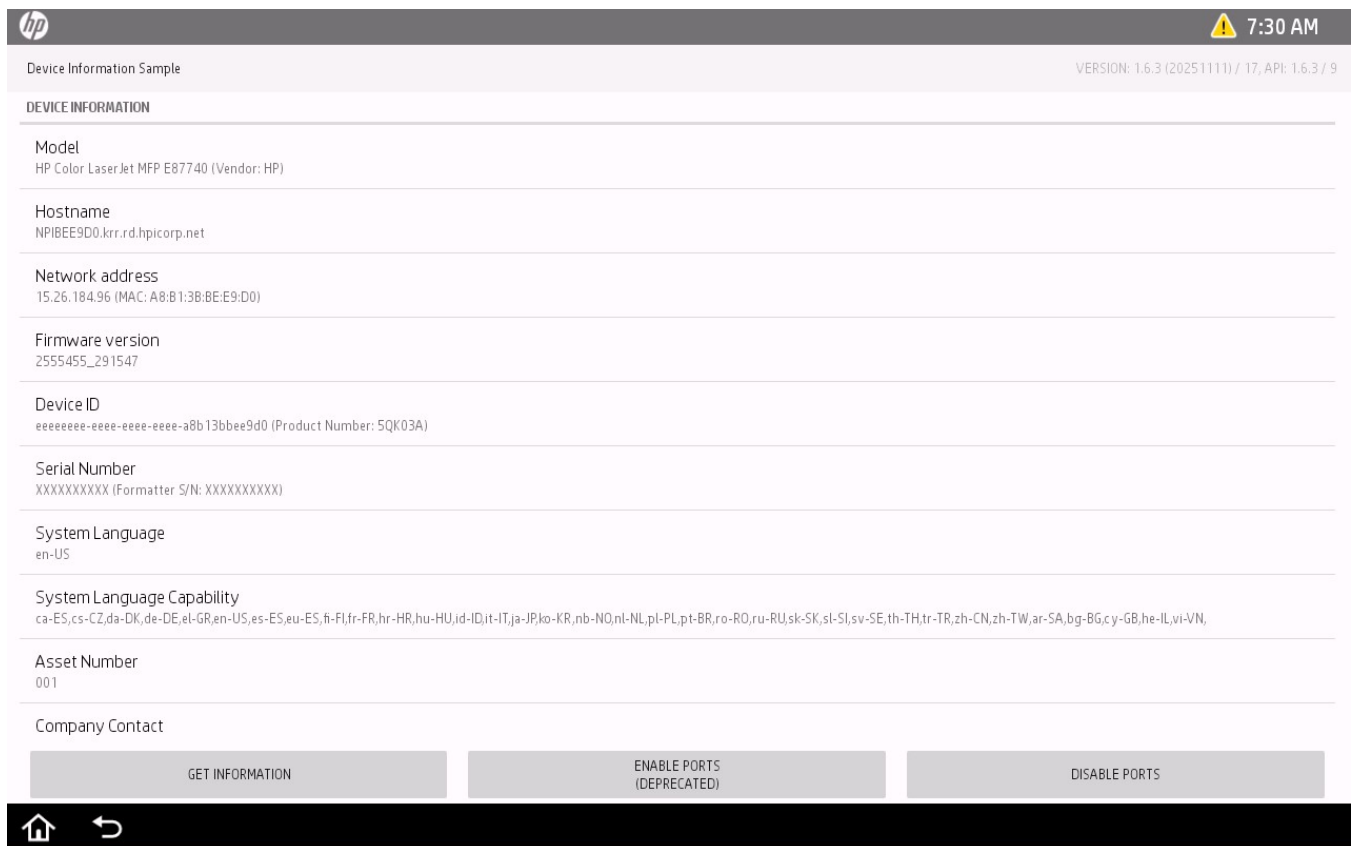
The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 DeviceInfo Sample on a device

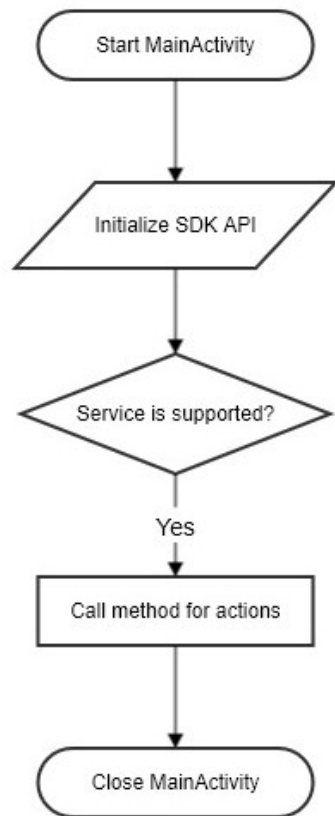
DeviceInfo Sample is a sample App that shows how to:

- Retrieve device information such as model, hostname, network address, firmware version, device id, serial number, system language, asset number, company contact, company name, device location, machine name and HP FutureSmart Level
- Disable ports for printing
- Enable port is deprecated



A main activity of DeviceInfo Sample

3 DeviceInfo Sample Flow



Flow of DeviceInfo Sample

4 Initializing SDK

Application requires to initialize the SDK services when starts with an activity for accessing the SDK services. The `com.hp.workpath.api.Workpath` includes `initialize()` method for this.

The sample shows how to initialize SDK.

1. Call `initialize()` method of Workpath SDK on resume event, and clear it on pause event. It is for calling initialization when an activity is switched. Refer to `MainActivity.java`.

```
1. @Override
2. protected void onResume() {
3.     super.onResume();
4.
5.     mContainer.setEnabled(false);
6.
7.     // call init task
8.     mInitializationTask = new InitializationTask(this);
9.     mInitializationTask.execute();
10. }
11.
12. @Override
13. protected void onPause() {
14.     super.onPause();
15.
16.     mInitializationTask.cancel(true);
17.     mInitializationTask = null;
18.
19.     ...
20. }
```

Initialization call in Activity Resume

2. Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, sdk exception or security exception will be thrown. Refer to `InitializationTask.java`.

```

1. @Override
2.     protected InitStatus doInBackground(final Void... params) {
3.         InitStatus status = InitStatus.NO_ERROR;
4.
5.         try {
6.             // initialize Workpath SDK
7.             Workpath.getInstance().initialize(mContextRef.get());
8.         } catch (SdkUnsupportedException sue) {
9.             Log.e(TAG, "SDK is not supported!", sue);
10.            mException = sue;
11.            status = InitStatus.INIT_EXCEPTION;
12.        } catch (SecurityException se) {
13.            Log.e(TAG, "Security exception!", se);
14.            mException = se;
15.            status = InitStatus.INIT_EXCEPTION;
16.        }
17.
18.        // Check if DeviceService is supported
19.        if (status == InitStatus.NO_ERROR
20.            && !DeviceService.isSupported(mContextRef.get())) {
21.            // DeviceService is not supported on this device
22.            status = InitStatus.NOT_SUPPORTED;
23.        }
24.
25.        return status;
26.    }

```

Exception handling by initializationTask

5 Checking DeviceService/DeviceSettingsService Capability

The `isSupported()` method returns the capability of each service. Refer to `MainActivity.java`.

```
1. mDisablePortsButton.setOnClickListener(new OnClickListener() {
2.     @Override
3.     public void onClick(View view) {
4.         if (!DeviceSettingsService.isSupported(MainActivity.this)) {
5.             showResult("DeviceSettingsService is not supported");
6.         } else {
7.             Result result = new Result();
8.             DeviceSettingsService.disableExternalPrinting(MainActivity.this, result);
9.
10.            if (result.getCode() == Result.RESULT_OK) {
11.                showResult("Ports disabled");
12.            } else {
13.                showResult("DeviceSettingsService.disableExternalPrinting()", result);
14.            }
15.        }
16.    }
17. });
```

Available service check by `isSupported()`

6 Getting device attributes

The device attributes have the keys to get the value. It's used with `DeviceService.getString()` for getting string types of data and `DeviceService.getStringArray()` for getting string array types of data. Each attribute must be queried by a method corresponding to the type of attribute. Otherwise exception will be thrown. Please see `DeviceAttribute` class for checking supported attributes.

The code snippet below shows example of such queries :

```

1.  if (sItemsToRefresh.contains(DeviceInfo.MODEL)) {
2.      Result vendorResult = new Result();
3.      String vendor = DeviceService.getString(mContextRef.get(), DeviceAttribute.DA_DEVICE
   E_VENDOR, vendorResult);
4.
5.      if (vendorResult.getCode() != RESULT_OK) {
6.          publishProgress("DeviceService.getString DA_DEVICE_VENDOR", vendorResult);
7.          vendor = VALUE_NA;
8.      }
9.
10.     Result modelNameResult = new Result();
11.     String modelName = DeviceService.getString(mContextRef.get(), DeviceInfo.MODEL.getA
   ttribute(), modelNameResult);
12.
13.     if (modelNameResult.getCode() != RESULT_OK) {
14.         publishProgress("DeviceService.getString DA_SYSTEM_MODELNAME", modelNameResult)
   ;
15.         modelName = VALUE_NA;
16.     }
17.
18.     String value = res.getString(R.string.model_value, modelName, vendor);
19.
20.     values.put(DeviceInfo.MODEL, value);
21. }
22.
23. if (sItemsToRefresh.contains(DeviceInfo.FIRMWARE_VERSION)) {
24.     Result result = new Result();
25.     String firmwareVersion = DeviceService.getString(mContextRef.get(), DeviceInfo.FIRM
   WARE_VERSION.getAttribute(), result);
26.
27.     if (result.getCode() != RESULT_OK) {
28.         publishProgress("DeviceService.getString DA_FIRMWARE_VERSION", result);
29.         firmwareVersion = VALUE_NA;
30.     }
31.
32.     values.put(DeviceInfo.FIRMWARE_VERSION, firmwareVersion);
33. }
34.
35. if (sItemsToRefresh.contains(DeviceInfo.DEVICE_ID)) {
36.     Result deviceIdResult = new Result();
37.     String deviceId = DeviceService.getString(mContextRef.get(), DeviceInfo.DEVICE_ID.g
   etAttribute(), deviceIdResult);
38.
39.     if (deviceIdResult.getCode() != RESULT_OK) {
40.         publishProgress("DeviceService.getString DA_SYSTEM_DEVICE_ID", deviceIdResult);
41.         deviceId = VALUE_NA;
42.     }

```

```
43.
44.     Result productNumberResult = new Result();
45.     String productNumber = DeviceService.getString(mContextRef.get(), DeviceAttribute.D
A_SYSTEM_PRODUCT_NUMBER, productNumberResult);
46.
47.     if (productNumberResult.getCode() != RESULT_OK) {
48.         publishProgress("DeviceService.getString DA_SYSTEM_PRODUCT_NUMBER", productNumb
erResult);
49.         productNumber = VALUE_NA;
50.     }
51.
52.     String value = res.getString(R.string.device_id_value, deviceId, productNumber);
53.     values.put(DeviceInfo.DEVICE_ID, value);
54. }
```

Reading the string type of DeviceAttribute in DeviceAttrReaderTask.java

7 Getting list of attributes

Some attributes represent the list of objects such as capabilities of supported languages from a device. These attribute details can be retrieved using `DeviceService.getStringArray()`.

The code snippet below shows example of such queries :

```

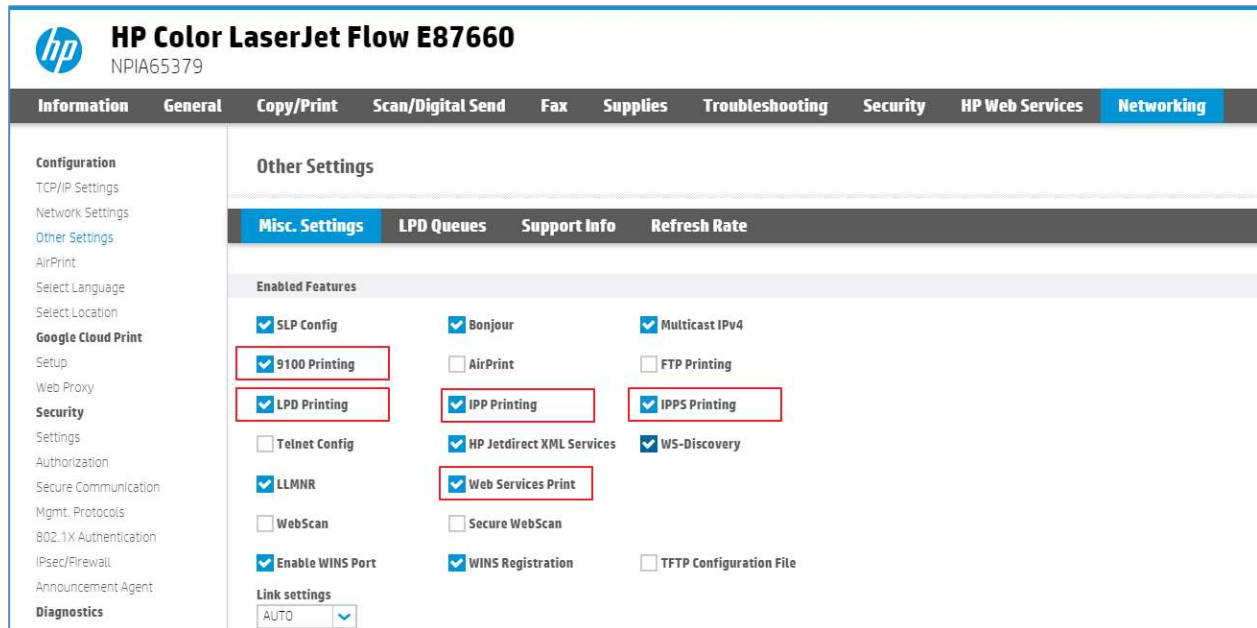
1. if (sItemsToRefresh.contains(DeviceInfo.SYSTEM_LANGUAGE_CAPABILITY)) {
2.     Result result = new Result();
3.     String[] systemLanguageCapability = DeviceService.getStringArray(mContextRef.get(),
        DeviceAttribute.DA_SYSTEM_LANGUAGE_CAPABILITY, result);
4.     StringBuilder systemLanguages = new StringBuilder(VALUE_NA);
5.     if (result.getCode() != RESULT_OK || systemLanguageCapability == null) {
6.         publishProgress("DeviceService.getStringArray DA_FEATURE_SYSTEM_LANGUAGE_CAPABI
LITY", result);
7.     } else {
8.         systemLanguages = new StringBuilder();
9.         for (String language : systemLanguageCapability) {
10.            systemLanguages.append(language).append(",");
11.        }
12.    }
13.    values.put(DeviceInfo.SYSTEM_LANGUAGE_CAPABILITY, systemLanguages.toString());
14. }

```

Reading a string array type of DeviceAttribute in DeviceAttrReaderTask.java

8 Enabling/Disabling ports for print

DeviceSettingsService provides the methods to handle the ports for print such as 9100 Printing, LPD Printing, IPP Printing, IPPS Printing and Web Services Print. The result of changes can check on "Networking>Configuration>Other Setting" of EWS.



EWS page for checking the port changes

DeviceInfo sample shows how to call DeviceSettingsService.

```

1. mDisablePortsButton.setOnClickListener(new OnClickListener() {
2.     @Override
3.     public void onClick(View view) {
4.         if (!DeviceSettingsService.isSupported(MainActivity.this)) {
5.             showResult("DeviceSettingsService is not supported");
6.         } else {
7.             Result result = new Result();
8.             DeviceSettingsService.disableExternalPrinting(MainActivity.this, result);
9.
10.            if (result.getCode() == Result.RESULT_OK) {
11.                showResult("Ports disabled");
12.            } else {
13.                showResult("DeviceSettingsService.disableExternalPrinting()", result);
14.            }
15.        }
16.    }
17. });

```

Calling DeviceSettingsService to disable port in MainActivity.java

8.1 Disable Port Permission

Starting from FutureSmart 5.9.2.2 / Workpath platform version 31.9, additional permission is required to use the `disableExternalPrinting()` API.

Without this permission, the `disableExternalPrinting()` API will not function.

```
1. <uses-permission android:name="com.hp.workpath.permission.admin.DISABLE_PRINTING_PORTS" />
```

Declare uses-permission in AndroidManifest.xml

For security reasons, the use of permission will be subject to Verification and Validation process (VAV Process) approval.

8.2 Deprecated `enableExternalPrinting()` API

```
1. DeviceSettingsService.enableExternalPrinting(MainActivity.this, result);
```

Deprecated `enableExternalPrinting()` API

This API has been deprecated. For security reasons, this functionality is no longer provided. Now, when this API is called, it always returns `Result.RESULT_OK`; however, the port is not actually enabled.

If you want to re-enable the port, you can do so through the EWS page mentioned above.

9 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

10 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.